Regex help sheet

Catherine Ives-Keeler

March 2026

Python Regular Expressions (REGEX) Reference Sheet

Introduction

A regular expression (REGEX) is a formal pattern specification language used to identify, extract, or manipulate structured substrings within unstructured text. It enables rule-based matching of character sequences using symbolic operators and quantifiers.

In text cleaning and preprocessing, regular expressions are particularly valuable because they allow systematic removal, transformation, or normalisation of noisy elements such as punctuation, numbers, URLs, and irregular spacing. This makes them a foundational tool in data preparation pipelines for natural language processing and statistical text analysis.

Core Python Functions

Function	Description
`re.search(pattern, string, flags)`	Returns the first match object. Returns `None` if no match is found.
`re.findall(pattern, string, flags)`	Returns a list of all matches. Returns an empty list if no match exists.
`re.sub(pattern, replacement, string, flags)`	Replaces all matches with `replacement`. Returns modified string.
`re.split(pattern, string, flags)`	Splits string at matches of pattern. Returns list of substrings.

Basic Python Usage Example

Before using regular expressions in Python, the `re` module must be imported.

```
import re

text = "The price is £45.50 and the code is AB123."

# Example 1: Find all numbers (including decimals)
numbers = re.findall(r"\d+\.?\d*", text)
print(numbers)

# Example 2: Replace digits with a placeholder
cleaned_text = re.sub(r"\d+", "[NUMBER]", text)
print(cleaned_text)

# Example 3: Search for a pattern
```

```
match = re.search(r"[A-Z]{2}\d{3}", text)

if match:
    print("Found code:", match.group())
```

Key observations:

- `r"..."` denotes a raw string, which prevents Python from interpreting backslashes.
- `re.findall()` returns all matches as a list.
- `re.sub()` replaces all matches.
- `re.search()` returns the first match object.

Anchors and Basic Symbols

Symbol	Meaning
`^`	Start of string
`$`	End of string
`.`	Any character except newline (`\n`)
`a|b`	Matches `a` or `b`
`\`	Escape special character
`ab`	Exact string match

Quantifiers

Pattern	Meaning
`*`	0 or more occurrences
`?`	0 or 1 occurrence
`+`	1 or more occurrences
`{5}`	Exactly 5 occurrences
`{5,10}`	Between 5 and 10 occurrences

Example:

```
xy*z
```

Matches: xz, xyz, xyyz, etc.

Character Classes

Pattern	Meaning
`\s`	Whitespace character
`\S`	Non-whitespace character
`\w`	Word character (letters, digits, underscore)
`\W`	Non-word character
`\d`	Digit
`\D`	Non-digit
`[xyz]`	x, y, or z
`[^xyz]`	Any character except x, y, or z

`[a-q]`	Character in range a to q
`[0-7]`	Digit in range 0 to 7

Special Characters

Pattern	Meaning
`\n`	Newline
`\t`	Tab
`\r`	Carriage return
`\xZZ`	Hexadecimal character
`\0`	Null character
`\v`	Vertical tab

Grouping and Assertions

Pattern	Meaning
`(xyz)`	Capturing group
`(?:xyz)`	Non-capturing group
`(?=xyz)`	Positive lookahead
`(?!xyz)`	Negative lookahead
`(?<=xyz)`	Positive lookbehind
`(?<!xyz)`	Negative lookbehind
`\b`	Word boundary

Replacement Patterns (Backreferences)

Pattern	Meaning
`$&`	Entire match
`$n`	nth captured group
`$‘`	Text before match
`$’`	Text after match

Flags

Flag	Meaning
`i`	Ignore case
`m`	Multi-line mode
`s`	Dot matches newline
`x`	Allow whitespace and comments in pattern
`u`	Unicode support

Practice Resources

- https://regex.sketchengine.co.uk/
- https://regexr.com/
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp